

PragatiFAQ

Phase-Aware FAQ Personalization

Feature Architecture, Design and Implementation Report

An Internship Project Documentation submitted for the

Vicharanashala Internship Program

Indian Institute of Technology Ropar

Submitted by

Vedha Shree J

Team Leader, CSFAQ (PragatiFAQ / Samagama Platform)

August 2026

Certificate

This is to certify that the project report titled "PragatiFAQ — Phase-Aware FAQ Personalization" is a bona fide record of the internship work carried out by Vedha Shree J as part of the Vicharanashala Internship Program, Indian Institute of Technology Ropar, on the CSFAQ / Samagama community FAQ platform. The design, implementation, and testing of the Phase-Aware FAQ Personalization feature described in this report were carried out under the guidance and supervision of the internship mentor(s), and represent an original contribution to the platform's codebase.

This report is submitted in partial fulfilment of the requirements of the internship program.

_____ Mentor / Guide Name

_____ Program Coordinator, Vicharanashala

_____ Date of Submission

Acknowledgement

I would like to express my sincere gratitude to the mentors and program coordinators of the Vicharanashala Internship Program at the Indian Institute of Technology Ropar for the opportunity to work on the CSFAQ / Samagama platform and for their continuous guidance, technical review, and encouragement throughout the course of this internship.

I am grateful to my fellow contributors on the CSFAQ project for maintaining a codebase and documentation set detailed enough to build on confidently, and for the review discussions that shaped the design of the Phase-Aware FAQ Personalization feature described in this report.

Finally, I thank the wider Samagama student community whose real usage patterns and feedback — the observation that students kept seeing FAQ content for internship phases they had already completed — directly motivated this feature.

Abstract

PragatiFAQ is the FAQ and community knowledge platform built for the Samagama student community under the Vicharanashala Internship Program at IIT Ropar. It gives students a centralized, verified knowledge base for technical queries, internship procedures, documentation and NOC requests, session logistics, and administrative announcements, backed by hybrid semantic + keyword search, an AI-assisted community question board, and an automated content ingestion and quality-control pipeline.

This report documents a feature designed and implemented on top of that platform: ‘Phase-Aware FAQ Personalization’. The Vicharanashala internship progresses through a fixed sequence of stages — General/Onboarding, Phase 1, Phase 2, Phase 3, and Completion — and different stages are relevant to different FAQ content. Before this feature, every student saw the entire FAQ corpus regardless of how far they had progressed, which meant a student who had finished Phase 1 still had to scroll past Phase 1 onboarding questions to find material relevant to their current stage.

The feature introduces a phase tag on every FAQ document, a per-student ‘InternshipProgress’ record tracking each student’s current phase, and a single reusable visibility rule that is applied at the database-query level everywhere FAQs are listed. The result is a FAQ page that automatically narrows itself to General plus the student’s current-and-upcoming phases, hides phases the student has already completed, and requires no changes to the platform’s existing search, category filtering, or pagination logic. An administrator interface allows staff to tag FAQ content by phase and to view or override any student’s recorded phase, with every override captured in the platform’s audit log.

The feature was verified with an automated integration test suite exercising every phase state — unauthenticated/guest, no-progress-record, Phase 1 through Completed — and confirming correct composition with the platform’s existing batch/program scoping so that phase filtering never leaks FAQ content across cohorts. This report covers the platform context, the problem motivating the feature, its architecture and detailed design, the full implementation across backend and frontend, the testing strategy and results, and directions for future enhancement.

Keywords: FAQ personalization, phase-aware content visibility, MongoDB/Mongoose, Express/TypeScript, React, internship knowledge platform, Samagama, PragatiFAQ.

Table of Contents

Right-click and choose "Update Field" (or press F9) to generate the table of contents with page numbers.

List of Figures and Tables appear inline within each chapter and are captioned in place.

Chapter 1 — Introduction

1.1 Background

The Vicharanashala Internship Program at the Indian Institute of Technology Ropar runs a multi-phase internship for a large, distributed cohort of students under the umbrella name "Samagama". As with any large student community, the same operational questions — how to submit a report, how session attendance works, how to raise a NOC request, what a particular error means, when the next Zoom session is — get asked repeatedly across chat groups and channels, and the answers are rarely searchable after the fact.

PragatiFAQ (internally also referred to as the Crowd Source FAQ platform, or by its codebase name "Yaksha FAQ Portal") was built to solve exactly this problem: a full-stack, self-maintaining FAQ and community knowledge portal that turns the community's accumulated conversations into a searchable, always-available knowledge base. It combines a hybrid vector-plus-keyword search engine, a community question-and-answer board with AI-assisted auto-answering, and a fully automated content ingestion pipeline that pulls transcripts from recorded Zoom sessions, extracts question-answer pairs with an LLM, and indexes them for retrieval within seconds.

This report documents a specific, self-contained feature designed and built on top of that existing platform during the internship: Phase-Aware FAQ Personalization. Sections 1.2 onward describe the platform context in enough depth to understand where the feature sits, and the remaining chapters (from Chapter 4 onward) go deep on the feature's own architecture, design, implementation, and testing — which is the primary deliverable of this report.

1.2 About the PragatiFAQ / Samagama Platform

PragatiFAQ is organized around four "zero-touch" pillars that describe how content enters, gets answered, stays correct, and is retired on the platform without requiring a human operator for the steady-state case:

- Zero-touch ingestion — Zoom meeting recordings, webhooks, and manual document uploads feed the knowledge base without a human having to schedule, categorize, or approve each item.
- Zero-touch answering — a 24-hour scheduler matches unanswered community posts against the existing knowledge base; high-confidence matches are posted automatically, low-confidence matches are queued for a human moderator.
- Zero-touch quality control — every approved FAQ is re-evaluated on a 6-hour cycle against the live knowledge base; drift, contradictions, and staleness are detected and flagged automatically for review.
- Zero-touch user lifecycle — account deletion anonymizes rather than destroys a user record, so reputation, attribution, and audit history remain intact for platform integrity.

The platform is designed to act as the operator for the steady-state case; people are reserved for handling genuine exceptions rather than routine, repetitive triage.

1.3 Purpose of this Report

The purpose of this report is threefold: first, to give enough platform context that the feature described here can be understood in its real operating environment rather than in isolation; second, to present a complete architectural and design account of the Phase-Aware FAQ Personalization feature; and third, to document its implementation, testing, and validation in enough technical depth that the work can be evaluated, reviewed, and extended by someone unfamiliar with the code.

1.4 Report Structure

The remainder of this report is organized as follows:

- Chapter 2 — the problem statement that motivated the feature.
- Chapter 3 — objectives and scope.
- Chapter 4 — platform-level system architecture and technology stack (context for the feature).
- Chapter 5 — requirements analysis (functional and non-functional).
- Chapter 6 — Feature Architecture: a deep architectural treatment of Phase-Aware FAQ Personalization, including component and sequence views.
- Chapter 7 — Detailed Design: data model, phase-visibility algorithm, and API contract.
- Chapter 8 — Implementation: backend, admin interface, and frontend integration, with annotated source code.
- Chapter 9 — Testing and Validation.
- Chapter 10 — Results and Discussion.
- Chapter 11 — Challenges Faced.
- Chapter 12 — Conclusion and Future Enhancements.
- References and Appendices.

Chapter 2 — Problem Statement

The Vicharanashala internship unfolds across a fixed sequence of stages: General/Onboarding, Phase 1, Phase 2, Phase 3, and Completion. Each stage has its own set of operationally relevant questions — onboarding has account setup and orientation questions, Phase 1 has its own submission and evaluation questions, and so on through Completion, which is concerned with certificates and closure procedures rather than ongoing project work.

Prior to this feature, PragatiFAQ's FAQ page showed the complete, undifferentiated FAQ corpus to every student regardless of how far they had progressed. A student who had already completed Phase 1 continued to see Phase 1 onboarding and submission questions mixed in with their current Phase 2 material, even though the Phase 1 content no longer applied to them in any way. This is a direct instance of a well-known content-management anti-pattern: personalization dimensions that exist in the data model (batch, category) but are not extended to the dimension that actually determines relevance for a returning user — their own progress.

2.1 Specific Issues Identified

- Information clutter — students had to scroll past FAQs that no longer applied to them to find what was actually relevant to their current stage, especially as the FAQ corpus grew over successive cohorts.
- Reduced trust in the FAQ page — an undifferentiated, growing list made it progressively harder for a student to judge which entries were current guidance and which were stale leftovers from an earlier stage they had already passed.
- A missing personalization dimension — FAQs were already scoped by program/batch and by category, but there was no mechanism to scope them by an individual student's own progress through the internship, even though that is the dimension a returning student cares about most.
- No lightweight way for administrators to reason about "who is at what stage" — phase information about students existed informally (if at all) and was not represented anywhere the FAQ system could act on it.

2.2 Why This Matters

A FAQ system is only as useful as the relevance of what it surfaces. As the corpus grows across multiple cohorts and multiple internship cycles, an undifferentiated list scales poorly — the signal-to-noise ratio for any individual student degrades even as the platform accumulates more useful content overall. Left unaddressed, this creates a perverse incentive: students learn to distrust or skip the FAQ page and fall back to asking in chat, which is precisely the behaviour PragatiFAQ exists to reduce.

Chapter 3 — Objectives and Scope

3.1 Objectives

The feature was designed to meet the following concrete objectives:

1. Tag every FAQ with the internship phase it belongs to — General, Phase 1, Phase 2, Phase 3, or Completion.
2. Track every student's current phase in the internship as first-class, queryable platform state.
3. Automatically filter the FAQ list shown to each student so that only FAQs relevant to their current and upcoming phases are displayed, while FAQs for phases they have already completed are hidden.
4. Make the integration minimal and additive — existing search, category filters, pagination, and the FAQ page's UI must continue to work exactly as before, operating on an already-narrowed result set rather than requiring their own logic changes.
5. Give administrators a simple, auditable interface to view and update each student's current phase, and to tag FAQ content by phase using the same workflow they already use to manage FAQs.
6. Preserve correctness of the platform's existing program/batch scoping — phase filtering must compose with batch scoping, never override or leak across it.

3.2 Scope

In scope for this feature:

- A new phase field on the FAQ model, and a new InternshipProgress collection tracking one phase value per student.
- A single, reusable phase-visibility rule (`getVisiblePhases`) applied consistently at every FAQ listing endpoint.
- Backend enforcement — the filter is applied at the MongoDB query level, not as a client-side/UI-only filter, so it cannot be bypassed by a modified frontend request.
- An admin dashboard tab for viewing and updating each student's phase, with every change written to the platform's existing admin audit log.
- A phase selector added to the existing FAQ create/edit admin forms.
- Automated integration tests covering every phase state and its composition with batch/program scoping.

Explicitly out of scope for this iteration (see also Chapter 12, Future Enhancements):

- Automatic phase advancement driven by milestone events (e.g. report submission, evaluation sign-off) — phase is set manually by an administrator in this version.
- Phase-aware ranking within search results (weighting matches toward the student's current phase) — the current implementation narrows the result set but does not re-rank within it.
- Bulk/CSV phase-tagging tools for retroactively classifying a large existing FAQ corpus.

Chapter 4 — Platform Architecture and Technology Stack

This chapter summarizes the architecture of the wider PragatiFAQ / Samagama platform that the Phase-Aware FAQ Personalization feature was built into, so that the feature-specific chapters that follow can be read in context. It is deliberately kept brief — architectural depth is reserved for Chapter 6, which covers the feature itself.

4.1 High-Level Architecture

The platform follows a conventional three-tier web architecture:

- Presentation tier — a React 18 + TypeScript single-page application built with Vite, styled with Tailwind CSS and Framer Motion, communicating with the backend over a versioned REST API via an Axios client.
- Application tier — a Node.js + Express 4 REST API written in TypeScript (ESM modules), organized into feature modules (auth, faq, community, search, support, admin, internship, and others), each exposing routes, controllers, and Mongoose models.
- Data tier — MongoDB Atlas (with native Atlas Vector Search for semantic embeddings) as the system of record, with an optional Upstash Redis layer and an in-process LRU cache for search-result caching, and Cloudinary for media/object storage.

4.2 Technology Stack

Layer	Technologies
Frontend	React 18, Vite, TypeScript, Tailwind CSS, Framer Motion, Axios, Recharts, React Router 6, Vitest
Backend	Node.js 22, Express 4, TypeScript (ESM), Mongoose 8, JWT, bcryptjs, Helmet, CORS, Morgan, Multer, Zod, express-rate-limit, dotenv, Vitest
Database & Storage	MongoDB Atlas (with Vector Search), Upstash Redis (optional), in-process LRU cache, Cloudinary
Search & AI	Local/hosted sentence embeddings, Atlas \$vectorSearch, MongoDB \$text search, Reciprocal Rank Fusion (RRF) merge
AI Providers	Anthropic, OpenAI, XAI, MiniMax — configurable per pipeline
DevOps & Observability	Sentry, Ngrok (local webhook tunnel), Twilio (SMS), SMTP, Prometheus-style metrics, structured file logging

Table 4.1 — Platform technology stack by layer.

Exact versions in use at the time of this report, taken directly from the workspace package manifests:

Package	Version
node (runtime)	v22.x
react	^18.2.0
vite	^5.4.21
typescript	^6.0.3
express	^4.18.2
mongoose	^8.0.3
zod	^4.4.3
tailwindcss	^3.4.0
framer-motion	^12.40.0

Table 4.2 — Key dependency versions, from apps/backend/package.json and apps/frontend/package.json.

4.3 Backend Module Layout

The backend is organized as feature modules rather than a single flat MVC tree. Each module owns its own Mongoose model(s), a controller layer, an Express router, and (where relevant) validation schemas. The Phase-Aware FAQ Personalization feature adds one new module, internship, alongside the existing modules:

```
apps/backend/src/modules/
|-- auth/           # registration, login, sessions, user model
|-- faq/            # FAQ CRUD, listing, search-matching, freshness <-- extended
|   |-- faq.model.ts
|   |-- faq.controller.ts
|   |-- faq.routes.ts
|   |-- __tests__/faq-phase-scoping.test.ts           <-- new
|-- internship/     # NEW MODULE -- phase tracking for this feature
|   |-- internship.model.ts
|   |-- internship.controller.ts
|   |-- internship.routes.ts
|   |-- phase.helper.ts
|-- admin/         # admin dashboard endpoints, admin-log.model.ts <-- extended
|-- community/     # community Q&A posts, comments, voting
|-- program/       # Batch / Category / app-settings (program scoping)
|-- ... (search, support, moderation, notifications, zoom, etc.)
```

Figure 4.1 — Backend module layout, showing where the new internship module sits alongside the extended faq and admin modules.

4.4 Existing Program/Batch Scoping

FAQs in PragatiFAQ are already scoped by a Batch (representing a cohort, term, or program) via a withProgramScope() helper applied to every FAQ query. This existing scoping mechanism is important context for the feature described in this report: phase visibility had to compose with batch scoping rather than replace or bypass it, so that a student never sees FAQ content belonging to a different program even if the phase filter alone would otherwise have allowed it through. This composition is verified directly in the test suite (see Section 9.4).

Chapter 5 — Requirements Analysis

5.1 Functional Requirements

ID	Requirement
FR-1	Every FAQ record shall carry a phase attribute with one of five fixed values: GENERAL, PHASE_1, PHASE_2, PHASE_3, COMPLETED.
FR-2	The system shall maintain one InternshipProgress record per student, tracking that student's current phase.
FR-3	A student with no InternshipProgress record shall be treated as being at the start of the internship (GENERAL) and see the full onboarding-through-Phase-3 visible set.
FR-4	An unauthenticated (guest) FAQ request shall default to the same start-of-internship visibility as a student with no progress record.
FR-5	The FAQ listing endpoints (grouped list and paginated list) shall return only FAQs whose phase is in the visible-phase set computed for the requesting user.
FR-6	A student who has reached COMPLETED shall see only GENERAL and COMPLETED FAQs.

ID	Requirement
FR-7	Administrators shall be able to view every registered student together with their current phase, filterable by phase.
FR-8	Administrators shall be able to update any student's current phase, and every such update shall be written to the admin audit log with the previous and new value.
FR-9	Administrators shall be able to set or change the phase tag of any FAQ from the existing FAQ create/edit interface.
FR-10	Phase filtering shall compose correctly with existing batch/program and category scoping without leaking FAQ content across programs.

Table 5.1 — Functional requirements.

5.2 Non-Functional Requirements

ID	Requirement	Rationale
NFR-1	Phase filtering must be enforced server-side at the database query level.	A client-side-only filter could be bypassed by directly calling the API, exposing content prematurely.
NFR-2	The feature must not require any change to the frontend search, pagination, or category-filter logic.	Minimizes regression risk on a widely used, already-tested page.
NFR-3	The visibility rule must be implemented once and reused everywhere FAQs are listed.	Prevents drift between the grouped-list and paginated-list endpoints producing inconsistent results.
NFR-4	Every admin override of a student's phase must be auditable.	Accountability for manual data changes affecting what a student can see.
NFR-5	The feature must introduce no new runtime dependency.	Keeps the platform's dependency surface and operational footprint unchanged.
NFR-6	The feature must be covered by automated tests exercising every phase state.	Phase logic is boundary-heavy (off-by-one risk at the first/last phase) and regressions must be caught before release.

Table 5.2 — Non-functional requirements.

5.3 Feasibility Analysis

Technical feasibility was high: the platform already used MongoDB/Mongoose with an established pattern for query-level scoping (with `ProgramScope` for batches), so adding a second, composable scope dimension (phase) followed an existing, proven pattern rather than introducing a new one. No new infrastructure, third-party service, or paid dependency was required. Operationally, the only new manual step introduced is an administrator setting a student's phase — a low-frequency action (once per phase transition per student) well suited to a simple dashboard table rather than a bespoke workflow tool.

Chapter 6 — Feature Architecture: Phase-Aware FAQ Personalization

This chapter is the architectural core of the report. It presents the feature as a system in its own right: its components, how they fit into the existing request pipeline, the data it owns, and the request-by-request behaviour it produces — before Chapter 7 goes into the detailed design of each piece and Chapter 8 into the concrete implementation.

6.1 Architectural Goals

Four goals shaped every architectural decision in this feature:

- Server-side truth — visibility must be decided by the backend from data it owns (the student's stored phase), never inferred or trusted from client input.
- Single source of the rule — exactly one function decides "which phases can this student see", called from every endpoint that lists FAQs, so the grouped list and the paginated list can never disagree.
- Composability — the phase filter must be an additional MongoDB query clause layered on top of the existing program/category filters, not a parallel or competing filtering path.
- Additivity — no existing model, endpoint, or UI component should need to change its own logic; they should simply operate on a result set that is already correctly scoped by the time it reaches them.

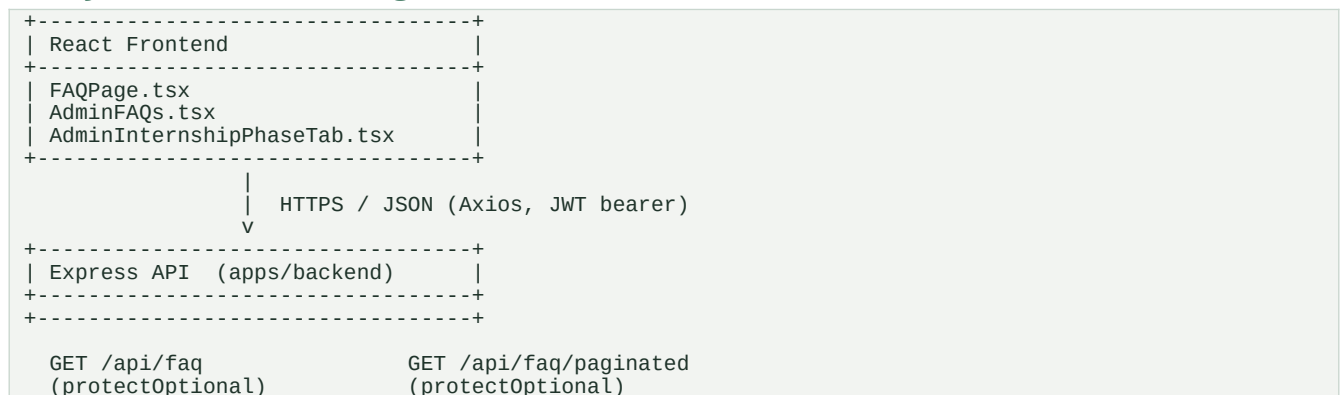
6.2 Component View

The feature is composed of four cooperating components, three new and one modified existing component:

Component	Type	Responsibility
InternshipProgress model	New (internship module)	Stores exactly one currentPhase value per student (userId is unique-indexed).
getVisiblePhases() helper	New (internship module)	Pure function: given a phase string, returns the ordered list of phases a student at that phase is allowed to see.
Internship admin API + UI	New (internship module + AdminInternshipPhaseTab)	Lets an administrator list every student's phase and update it, with audit logging.
FAQ model + controller	Modified (faq module)	FAQ gains a phase field; the two listing controllers (getAllFAQs, getPaginatedFAQs) resolve the requester's visible phases and add them to the existing Mongo query.

Table 6.1 — Feature components and responsibilities.

6.3 System Context Diagram



```

    |-----+-----|
    +-----+-----+
    |
    |   v
faq.controller.ts : getAllFAQs() / getPaginatedFAQs()
  1. withProgramScope(query, batchId)
  2. look up InternshipProgress for req.user
  3. getVisiblePhases(currentPhase) --> phase.helper.ts
  4. scoped.phase = { $in: visiblePhases }
  5. FAQ.find(scoped) / countDocuments
    |
    |   v
MongoDB collection: yaksha_faq_faqs
(fields used: phase, phaseOrder, batchId, status)
    |
    |   v
PUT /api/admin/internship/:userId (protect + adminOnly)
    |
    |   v
internship.controller.ts
  -> InternshipProgress.findOneAndUpdate(..., { upsert: true })
  -> AdminLog.create(action: 'internship_phase_override')
  
```

Figure 6.1 — System context: request flow for FAQ listing (student-facing) and phase override (admin-facing).

6.4 Sequence: Student Loads the FAQ Page

The following sequence shows the exact runtime path taken when a logged-in student opens the FAQ page, illustrating how phase scoping is layered onto the existing request without disturbing it:

1. The React FAQPage component calls GET /api/faq (optionally with a batchId query parameter), sending the student's JWT if one is present.
2. The route is bound to protectOptional rather than protect — a valid token attaches req.user, a missing token lets the request continue as a guest instead of being rejected.
3. Inside getAllFAQs, the handler first builds the existing program-scoped Mongo query via withProgramScope(query, batchIdFromQuery(req)) exactly as it did before this feature.
4. If req.user is present, the handler looks up that user's InternshipProgress document by userId; if absent (guest) or if no document exists, currentPhase defaults to GENERAL.
5. getVisiblePhases(currentPhase) computes the ordered list of visible phase values for that student.
6. The handler adds phase: { \$in: visiblePhases } to the already program-scoped query object.
7. FAQ.countDocuments(scoped) and the subsequent find/aggregate run against this combined filter, so the database itself only ever returns FAQs the student is entitled to see.
8. The response is grouped by category and paginated exactly as before — these downstream steps are completely unaware that phase scoping happened; they simply see a smaller, already-correct result set.

6.5 Sequence: Administrator Updates a Student's Phase

1. An administrator opens the "Internship Phase" tab inside AdminWelcomePage, which renders AdminInternshipPhaseTab and fetches GET /api/admin/internship, returning every registered student joined with their current phase (defaulting unset students to GENERAL in the response, without requiring a document to exist for every user).
2. The administrator selects a student, chooses a new phase in the edit modal, and submits — the frontend calls PUT /api/admin/internship/:userId with { currentPhase }.
3. The route is protected by protect + adminOnly, so only authenticated admin/moderator accounts can reach it.
4. The controller validates currentPhase against the fixed enum, loads the previous value (if any) for the audit trail, and performs an atomic InternshipProgress.findOneAndUpdate(..., { upsert: true }) — creating the record on first use or updating it thereafter.

5. An AdminLog entry is written with action: `internship_phase_override`, the target user id, and a `changes[]` array recording the old and new `currentPhase`, giving a permanent, queryable audit trail of every manual phase change.
6. The very next FAQ request that student makes is scoped by the new phase — there is no caching layer between `InternshipProgress` and the FAQ query, so the change takes effect immediately.

6.6 Deployment / Runtime View

The feature introduces no new deployable unit, no new external service, and no new environment variable. It runs inside the same Express process and against the same MongoDB Atlas cluster as the rest of the platform, adding one new collection (`internshipprogresses`) and two new indexed fields on the existing FAQ collection (`phase`, `phaseOrder`). This keeps the feature’s operational footprint at zero incremental infrastructure cost — a deliberate architectural choice consistent with NFR-5 (Section 5.2).

Chapter 7 — Detailed Design

7.1 Data Model

Two data-model changes support this feature, both consistent with the platform’s existing MongoDB / Mongoose schema conventions.

7.1.1 FAQ — extended with a phase tag

The existing FAQ document gains two new fields:

Field	Type	Default	Purpose
<code>phase</code>	String enum: <code>GENERAL</code> , <code>PHASE_1</code> , <code>PHASE_2</code> , <code>PHASE_3</code> , <code>COMPLETED</code>	<code>'GENERAL'</code>	The internship stage this FAQ is relevant to.
<code>phaseOrder</code>	Number (indexed)	0	Reserved for a future fine-grained ordering of FAQs within a phase; not yet consumed by any query in this release (see Section 12.1).

Table 7.1 — New fields on the FAQ schema.

7.1.2 InternshipProgress — new collection

A new collection records exactly one phase value per student:

Field	Type	Notes
<code>userId</code>	ObjectId, ref: User	Required, unique + indexed — enforces one progress record per user.
<code>currentPhase</code>	String enum (same 5 values)	Defaults to <code>'GENERAL'</code> .
<code>createdAt</code> / <code>updatedAt</code>	Date	Mongoose timestamps.

Table 7.2 — `InternshipProgress` schema.

7.1.3 Entity Relationship View

+-----+ User (existing) +-----+	
_id	



Figure 7.1 — Entity relationships added or extended by this feature (new elements marked NEW).

7.2 The Phase-Visibility Algorithm

The heart of the feature is a single pure function, `getVisiblePhases(phase)`, that maps a student’s current phase to the ordered list of phase values that student is allowed to see. The rule is: always include GENERAL, plus the student’s current phase and every phase after it; phases strictly before the student’s current one are excluded because that content is already resolved for them.

currentPhase input	Visible phases returned
(none / unset / GENERAL)	GENERAL, PHASE_1, PHASE_2, PHASE_3
PHASE_1	GENERAL, PHASE_1, PHASE_2, PHASE_3
PHASE_2	GENERAL, PHASE_2, PHASE_3
PHASE_3	GENERAL, PHASE_3
COMPLETED	GENERAL, COMPLETED

Table 7.3 — Visibility truth table for every phase state (matches the automated test matrix in Section 9.3).

Two edge cases are handled explicitly and deliberately, rather than falling out of the general case by accident:

- A brand-new student (no `InternshipProgress` record) is treated identically to a GENERAL student — they see the full onboarding-through-Phase-3 range, since they have not "completed" anything yet.
- A COMPLETED student is treated as a terminal state — they no longer see any of PHASE_1/2/3 (all resolved), and instead of exposing an out-of-range slice they see exactly GENERAL + COMPLETED.

These two cases are the reason the function branches on `currentIndex === 0` and `currentIndex === PHASE_ORDER.length - 1` rather than using a single slice expression for every input — a naive

slice(currentIndex) would have included COMPLETED for every earlier phase too, which is incorrect (a Phase 2 student should not see completion-closure content).

7.3 API Contract

7.3.1 Student-Facing Endpoints (modified)

Method & Path	Auth	Change
GET /api/faq	protectOptional (was public, unauthenticated-only)	Response now additionally scoped by the caller's visible phases.
GET /api/faq/paginated	protectOptional (was public, unauthenticated-only)	Same phase scoping applied before pagination.

Table 7.4 — Modified public FAQ endpoints. Neither endpoint changed its request or response shape — only the underlying result set changed.

7.3.2 Admin Endpoints (new)

Method & Path	Auth	Request body	Response
GET /api/admin/internship	protect + adminOnly	—	Array of { _id, name, email, currentPhase } for every user with role 'user'.
PUT /api/admin/internship/:userId	protect + adminOnly	{ currentPhase: 'GENERAL' 'PHASE_1' 'PHASE_2' 'PHASE_3' 'COMPLETED' }	The updated (or newly created) InternshipProgress document.

Table 7.5 — New internship administration endpoints, mounted at /api/admin/internship.

7.3.3 FAQ Create/Update Payload Extension

The existing createFAQ and updateFAQ payloads gain one optional field:

```
POST /api/faq
{
  "question": "How do I raise a NOC request in Phase 2?",
  "answer": "...",
  "category": "Documentation",
  "batchId": "665f1c2e9a1b2c3d4e5f6789",
  "phase": "PHASE_2"           // optional, defaults to "GENERAL"
}

PUT /api/faq/:id
{
  "phase": "PHASE_3"           // optional – only sent if changing the tag
}
```

Chapter 8 — Implementation

This chapter walks through the actual implementation module by module, in the order a reviewer would naturally read the code: the new internship module first, then the changes made to the existing faq module to consume it, then the admin and student-facing frontend integration.

8.1 The internship Module

8.1.1 InternshipProgress Model

apps/backend/src/modules/internship/internship.model.ts:

```
export type InternshipPhase =
  | "GENERAL" | "PHASE_1" | "PHASE_2" | "PHASE_3" | "COMPLETED";

export interface IInternshipProgress extends Document {
```



```

    userId: Types.ObjectId;
    currentPhase: InternshipPhase;
    createdAt: Date;
    updatedAt: Date;
  }

  const InternshipProgressSchema = new Schema(
    {
      userId: {
        type: Schema.Types.ObjectId,
        ref: "User",
        required: true,
        unique: true,
        index: true,
      },
      currentPhase: {
        type: String,
        enum: ["GENERAL", "PHASE_1", "PHASE_2", "PHASE_3", "COMPLETED"],
        default: "GENERAL",
      },
    },
    { timestamps: true }
  );

  export default mongoose.model<IInternshipProgress>(
    "InternshipProgress", InternshipProgressSchema
  );

```

The `unique: true` constraint on `userId` is the schema-level guarantee behind FR-2 — it is physically impossible for a student to end up with two conflicting phase records, which means the controller can safely use an upsert (Section 8.1.3) without a separate existence check first.

8.1.2 Phase Visibility Helper

`apps/backend/src/modules/internship/phase.helper.ts` — the single function described in Section 7.2:

```

const PHASE_ORDER = ['GENERAL', 'PHASE_1', 'PHASE_2', 'PHASE_3', 'COMPLETED'];

/**
 * A student sees GENERAL FAQs plus every phase from their current one
 * onward. Phases before their current one are already completed and
 * are hidden. A COMPLETED student only sees GENERAL + COMPLETED.
 */
export function getVisiblePhases(phase: string) {
  const index = PHASE_ORDER.indexOf(phase);
  const currentIndex = index === -1 ? 0 : index;

  if (currentIndex === 0) {
    return ['GENERAL', 'PHASE_1', 'PHASE_2', 'PHASE_3'];
  }

  if (currentIndex === PHASE_ORDER.length - 1) {
    return ['GENERAL', 'COMPLETED'];
  }

  return ['GENERAL', ...PHASE_ORDER.slice(currentIndex, PHASE_ORDER.length - 1)];
}

```

Because this function takes a plain string and returns a plain array with no database or request dependency, it is trivially unit-testable in isolation and is exactly the kind of logic that benefits from being pulled out of the controller — every call site (currently two, potentially more in future) is guaranteed to apply an identical rule.

8.1.3 Internship Controller

`apps/backend/src/modules/internship/internship.controller.ts`:

```

// GET /api/admin/internship
export const listInternshipProgress = async (_req, res) => {
  const users = await User.find({ role: 'user' }, 'name email')
    .sort({ createdAt: -1 }).lean();
  const progresses = await InternshipProgress.find({}).lean();
  const byUserId = new Map(progresses.map(p => [p.userId.toString(), p.currentPhase]));

```

```

const result = users.map(u => ({
  _id: u._id, name: u.name, email: u.email,
  currentPhase: byUserId.get(u._id.toString()) ?? 'GENERAL',
}));
res.status(200).json(result);
};

// PUT /api/admin/internship/:userId
export const updateInternshipPhase = async (req, res) => {
  const { userId } = req.params;
  const { currentPhase } = req.body;
  const adminId = req.user?._id;

  if (!currentPhase || !VALID_PHASES.includes(currentPhase)) {
    return res.status(400).json({ message: `currentPhase must be one of ...` });
  }
  const user = await User.findById(userId);
  if (!user) return res.status(404).json({ message: 'User not found' });

  const previous = await InternshipProgress.findOne({ userId });
  const progress = await InternshipProgress.findOneAndUpdate(
    { userId }, { $set: { currentPhase } }, { new: true, upsert: true },
  );

  if (adminId) {
    await AdminLog.create({
      adminId, action: 'internship_phase_override',
      targetId: user._id, targetType: 'user',
      details: `Set internship phase for ${user.name} to ${currentPhase}`,
      changes: [{ field: 'currentPhase',
        oldValue: previous?.currentPhase ?? 'GENERAL',
        newValue: currentPhase }],
    });
  }
  res.status(200).json(progress);
};

```

listInternshipProgress deliberately does a left-join in application code (via a Map) rather than a Mongo \$lookup aggregation: the platform's user base for a single cohort is small enough that two flat queries plus an in-memory map are simpler to read, test, and maintain than an aggregation pipeline, at negligible performance cost. updateInternshipPhase records previous before the update specifically so the AdminLog entry captures a true before/after diff rather than just the new value.

8.1.4 Internship Routes

apps/backend/src/modules/internship/internship.routes.ts, mounted at /api/admin/internship in bootstrap/routes.ts:

```

const router = express.Router();
router.use(protect);
router.use(adminOnly);

router.get('/', listInternshipProgress);
router.put('/:userId', updateInternshipPhase);

export default router;

```

Both guards are applied at the router level with router.use(...) rather than repeated on each route — every current and future route added to this router is admin-only by construction, removing an entire class of "forgot to protect a new route" bugs.

8.2 Extending the faq Module

8.2.1 FAQ Model Changes

apps/backend/src/modules/faq/faq.model.ts — the phase and phaseOrder fields added to the existing schema:

```

export type InternshipPhase =

```

```

    | 'GENERAL' | 'PHASE_1' | 'PHASE_2' | 'PHASE_3' | 'COMPLETED';

// ...inside IFAQ interface:
phase: InternshipPhase;
phaseOrder: number;

// ...inside faqSchema definition:
phase: {
  type: String,
  enum: ["GENERAL", "PHASE_1", "PHASE_2", "PHASE_3", "COMPLETED"],
  default: "GENERAL"
},
phaseOrder: {
  type: Number,
  default: 0,
  index: true
},

```

The enum constraint is enforced at three independent layers for defence in depth: the Mongoose schema (rejects an invalid value at the database-write layer), the Zod request validator (Section 8.2.3, rejects it before it ever reaches the controller), and the fixed VALID_PHASES array checked explicitly in `updateFAQ`. Any one of the three would catch a malformed value; having all three means a bug in one layer cannot silently let bad data through.

8.2.2 FAQ Controller Changes

Both FAQ-listing endpoints received the identical three-line addition, placed immediately after the existing `withProgramScope()` call so that phase scoping is visibly layered on top of — not a replacement for — program scoping:

```

// getAllFAQs (apps/backend/src/modules/faq/faq.controller.ts)
const scoped = withProgramScope(query, batchIdFromQuery(req));

// Internship-phase scope: an intern sees GENERAL FAQs plus FAQs
// tagged for their current phase (from InternshipProgress). Guests
// (no req.user — this route has no `protect`) default to GENERAL.
const progress = req.user
  ? await InternshipProgress.findOne({ userId: req.user._id })
  : null;
const visiblePhases = getVisiblePhases(progress?.currentPhase ?? 'GENERAL');
(scoped as Record<string, unknown>).phase = { $in: visiblePhases };

const totalCount = await FAQ.countDocuments(scoped);
// ... existing grouping / pagination logic, unchanged

```

`getPaginatedFAQs` receives the same block ahead of its own `find/count` calls, guaranteeing the two endpoints can never disagree about what a given student is allowed to see.

`createFAQ` and `updateFAQ` were extended to accept and persist the optional phase field:

```

// createFAQ
const { question, answer, category, batchId: rawBatchId,
  freshnessTier, reviewIntervalDays, phase } = req.body;
...
const faq = await FAQ.create({
  question: question_, answer: answer_, category: category_,
  batchId: new Types.ObjectId(batchId),
  phase: phase ?? 'GENERAL',
  freshnessTier: tier, reviewIntervalDays: interval,
});
...
});

// updateFAQ
if (phase && ['GENERAL', 'PHASE_1', 'PHASE_2', 'PHASE_3', 'COMPLETED'].includes(phase)) {
  faq.phase = phase;
}

```

8.2.3 Validation Layer

`apps/backend/src/utls/auth/validation.ts` — the Zod schemas for FAQ create/update both gained an optional phase enum, so a malformed value is rejected with a 400 before the controller runs:

```
export const createFAQSchema = z.object({
  ...
  phase: z.enum(['GENERAL', 'PHASE_1', 'PHASE_2', 'PHASE_3', 'COMPLETED']).optional(),
});

export const updateFAQSchema = z.object({
  ...
  phase: z.enum(['GENERAL', 'PHASE_1', 'PHASE_2', 'PHASE_3', 'COMPLETED']).optional(),
});
```

8.2.4 Route-Level Auth Change — protectOptional

The FAQ listing routes previously had no auth middleware at all, since they were purely public. To resolve a student's phase, the controller needs `req.user` when a token is present but must not reject requests without one — so the routes were switched from no middleware to the platform's existing `protectOptional` middleware (already used elsewhere on the platform for a similar "identify if possible, never block" case), rather than introducing a new one:

```
// apps/backend/src/modules/faq/faq.routes.ts
router.get('/', protectOptional, getAllFAQs);
router.get('/paginated', protectOptional, getPaginatedFAQs);
```

`protectOptional` decodes a bearer token and attaches `req.user` exactly like the strict `protect` middleware when a token is present and valid, but simply calls `next()` with `req.user` left undefined when no token is present at all — it only rejects a request that presented a token that turned out to be invalid, expired, or revoked. This preserves the fully anonymous, no-login-required browsing path (public guests still get a sensible GENERAL-onward default) while unlocking per-student personalization the moment a user is logged in.

8.2.5 Admin Audit Log Extension

`apps/backend/src/modules/admin/admin-log.model.ts` — a new `AdminAction` value was added so phase overrides appear in the same audit trail as every other administrative action:

```
export type AdminAction =
  | 'approve_faq' | 'reject_faq' | 'edit_faq' | 'delete_faq'
  | 'create_faq' | 'login' | 'settings_update'
  | 'onboarding_override'
  | 'internship_phase_override'; // <-- added
```

8.2.6 Route Registration

`apps/backend/src/bootstrap/routes.ts` — two lines wire the new module into the running server:

```
import adminInternshipRoutes from '../modules/internship/internship.routes.js';
...
router.use('/admin/internship', adminInternshipRoutes);
```

8.3 Frontend Integration

8.3.1 Admin — Internship Phase Tab

A new tab, `AdminInternshipPhaseTab.tsx`, was added to the existing admin welcome/management dashboard (`AdminWelcomePage.tsx`), alongside its existing tabs (Orientation, Onboarding, Journeys, Mentors, Audit, Resources, and so on). It lists every registered student with their current phase, supports filtering the list by phase, and opens a modal to change a selected student's phase:

```
export default function AdminInternshipPhaseTab() {
  const [users, setUsers] = useState<InternshipUser[]>([]);
  const [filter, setFilter] = useState<'all' | InternshipPhase>('all');
  const [editingUser, setEditingUser] = useState<InternshipUser | null>(null);

  const fetchData = async () => {
    const res = await adminApi.get('/admin/internship');
    setUsers(res.data);
  };
}
```

```

useEffect(() => { fetchData(); }, []);

const handleUpdate = async (e: React.FormEvent) => {
  e.preventDefault();
  await adminApi.put(`/admin/internship/${editingUser._id}`, {
    currentPhase: editingUser.currentPhase,
  });
  setEditingUser(null);
  fetchData(); // re-pull the list so the table reflects the change
};
// ...renders a filterable table + an edit modal (see Fig. 8.1)
}

```

The tab was registered in `AdminWelcomePage.tsx` next to the existing tab set:

```

import AdminInternshipPhaseTab from '../components/welcome/AdminInternshipPhaseTab';

const [activeTab, setActiveTab] = useState<
  'orientation' | 'onboarding' | 'projects' | 'timeline' | 'mentors'
  | 'audit' | 'resources' | 'journeys' | 'internship' // <-- added
>('orientation');
...
{activeTab === 'internship' && <AdminInternshipPhaseTab />}

```

8.3.2 Admin — FAQ Management Form

The existing FAQ management screen, `AdminFAQs.tsx`, was extended in three places using the same patterns already used for its other fields (e.g. `freshnessTier`): a new column in the FAQ table, and a phase `<select>` in both the "create FAQ" and "edit FAQ" modals, defaulting to General:

```

<div>
  <label className={adminLabel}>Internship Phase</label>
  <select
    value={editFaq.phase ?? 'GENERAL'}
    onChange={e => setEditFaq(f => f ? { ...f, phase: e.target.value as FAQ['phase'] } : null)}
    className={`${adminSelect} w-full`}
  >
    <option value="GENERAL">General</option>
    <option value="PHASE_1">Phase 1</option>
    <option value="PHASE_2">Phase 2</option>
    <option value="PHASE_3">Phase 3</option>
    <option value="COMPLETED">Completed</option>
  </select>
</div>

```

Administrators therefore tag FAQ content by phase using the exact same create/edit workflow they already use for every other FAQ attribute — no separate "phase management" screen was introduced, honouring the additivity goal from Section 6.1.

8.3.3 Student-Facing FAQ Page

`FAQPage.tsx` required no logic changes at all. It calls `GET /api/faq` with the same parameters as before; the response it receives is simply pre-narrowed to the requesting student's visible phases by the time it arrives. Its existing search box, category pill filters, and pagination controls all continue to operate purely on the response payload, unaware that a phase filter was ever applied upstream — which is exactly the additive integration described as objective 4 in Section 3.1.

8.4 Summary of Files Touched

File	Type of change
<code>apps/backend/src/modules/internship/internship.model.ts</code>	New
<code>apps/backend/src/modules/internship/phase.helper.ts</code>	New
<code>apps/backend/src/modules/internship/internship.controller.ts</code>	New
<code>apps/backend/src/modules/internship/internship.routes.ts</code>	New

File	Type of change
apps/backend/src/modules/faq/__tests__/faq-phase-scoping.test.ts	New
apps/backend/src/modules/faq/faq.model.ts	Modified — phase, phaseOrder fields
apps/backend/src/modules/faq/faq.controller.ts	Modified — phase scoping in list endpoints, phase in create/update
apps/backend/src/modules/faq/faq.routes.ts	Modified — protectOptional on public list routes
apps/backend/src/utis/auth/validation.ts	Modified — phase in create/update Zod schemas
apps/backend/src/modules/admin/admin-log.model.ts	Modified — new AdminAction value
apps/backend/src/bootstrap/routes.ts	Modified — mount internship router
apps/frontend/src/admin/components/welcome/AdminInternshipPhaseTab.tsx	New
apps/frontend/src/admin/pages/AdminWelcomePage.tsx	Modified — new tab registration
apps/frontend/src/admin/pages/AdminFAQs.tsx	Modified — phase column + selector
apps/frontend/src/pages/FAQPage.tsx	Unaffected in logic — consumes the already-scoped response

Table 8.1 — Complete file-change footprint of the feature: 5 new files, 9 modified files.

Chapter 9 — Testing and Validation

9.1 Test Strategy

Because the feature’s correctness lives almost entirely in a database-query composition (program scope + phase scope) rather than in isolated pure logic, the test suite is written as integration tests against a real (in-memory) MongoDB instance rather than as mocked unit tests. This choice was deliberate: a mocked test would have to re-encode the exact Mongo query shape being asserted on, which risks the test and the implementation drifting together and passing even if the query semantics were subtly wrong. Running the actual controller against a real MongoDB engine (via `mongodb-memory-server`) exercises the genuine \$in query and genuine index behaviour.

apps/backend/src/modules/faq/__tests__/faq-phase-scoping.test.ts uses Vitest with MongoMemoryServer:

```
beforeAll(async () => {
  mongo = await MongoMemoryServer.create();
  await mongoose.connect(mongo.getUri());
}, 120_000);

afterAll(async () => {
  await mongoose.disconnect();
  await mongo.stop();
});

async function seedFaqsAcrossPhases() {
  await FAQ.create([
    { question: 'General Q', answer: 'A', category: 'GenCat', phase: 'GENERAL', status: 'approved' },
    { question: 'Phase 1 Q', answer: 'A', category: 'P1Cat', phase: 'PHASE_1', status: 'approved' },
    { question: 'Phase 2 Q', answer: 'A', category: 'P2Cat', phase: 'PHASE_2', status: 'approved' },
    { question: 'Phase 3 Q', answer: 'A', category: 'P3Cat', phase: 'PHASE_3', status: 'approved' },
    { question: 'Completed Q', answer: 'A', category: 'CompCat', phase: 'COMPLETED', status: 'approved' },
  ]);
}
```

Each test calls the real `getAllFAQs` controller with a hand-built Express-shaped request/response pair (`mockReq` / `mockRes`) and asserts on the set of categories present in the grouped response — categories are a convenient, readable proxy for "which phase buckets came back", since each seeded phase was given its own unique category.

9.2 Test Environment

Item	Detail
Test runner	Vitest
Database	mongodb-memory-server (in-process, ephemeral MongoDB instance — no external DB dependency)
Scope	<code>apps/backend/src/modules/faq/__tests__/faq-phase-scoping.test.ts</code>
Fixtures	Five FAQs, one per phase, each in a distinct category; reset with <code>deleteMany()</code> before every test via <code>beforeEach</code>

Table 9.1 — Test environment summary.

9.3 Test Cases and Results

#	Test case	Expected visible categories	Result
T1	Unauthenticated request (no req.user)	GenCat, P1Cat, P2Cat, P3Cat	Pass
T2	Authenticated user, no InternshipProgress record	GenCat, P1Cat, P2Cat, P3Cat	Pass
T3	<code>currentPhase = PHASE_1</code>	GenCat, P1Cat, P2Cat, P3Cat	Pass
T4	<code>currentPhase = PHASE_2</code> (Phase 1 completed)	GenCat, P2Cat, P3Cat	Pass
T5	<code>currentPhase = PHASE_3</code> (Phase 2 completed)	GenCat, P3Cat	Pass
T6	<code>currentPhase = COMPLETED</code>	CompCat, GenCat	Pass
T7	Phase scoping composed with batch/program scoping across two batches	Program-A student at PHASE_2 sees Program A's Phase-2 category, not Program A's completed Phase-1 category, and not Program B's content at all	Pass

Table 9.2 — Test case matrix and results. All seven cases pass, matching the truth table in Table 7.3.

9.4 Cross-Program Isolation Test (T7) — Detail

Test T7 is the most architecturally significant test in the suite, because it directly validates NFR-1/FR-10 (Section 5): that phase scoping composes with, and never bypasses, the platform's existing program/batch boundary. It creates two batches (Program A, Program B), seeds Phase-1 and Phase-2 FAQs under Program A and a Phase-1 FAQ under Program B, sets a student to PHASE_2, and asserts three things simultaneously:

- The student's completed Phase 1 category within their own program (ACat1) is correctly hidden.
- Their current Phase 2 category within their own program (ACat2) is correctly shown.
- Program B's Phase 1 category (BCat1) never appears at all, regardless of phase — proving the two scopes are ANDed together, not ORed, and that phase scoping cannot leak content across a program boundary.

9.5 Recommended Manual Verification Checklist

Beyond the automated suite, the following manual walkthrough is recommended before this feature is signed off for release, and is the checklist an admin/QA reviewer can follow end-to-end using only the admin dashboard and the public FAQ page — no database access required:

1. Tag at least one FAQ under each of the five phases from the FAQ create/edit form (Section 8.3.2), in a single test batch.
2. Open the Internship Phase tab (Section 8.3.1) and set a test student account to each phase in turn, from GENERAL through COMPLETED.
3. After each change, load the FAQ page as that student and confirm the visible category list matches the truth table in Table 7.3 exactly.
4. Confirm each phase change produced exactly one new `internship_phase_override` entry in the admin audit log, with the correct old/new value pair.

This checklist is a direct, practical translation of the automated test matrix (Table 9.2) into steps a non-engineer reviewer can execute from the UI alone, and is the recommended acceptance procedure for this feature.

9.6 Regression Considerations

Because the feature changes the auth middleware and query construction on two heavily used public endpoints (GET `/api/faq` and GET `/api/faq/paginated`), any release checklist for this change should specifically re-run the platform's existing FAQ-related test coverage and end-to-end specs (tests/e2e/student.spec.ts, tests/e2e/admin.spec.ts) to confirm the search, category-filter, and pagination behaviour on the FAQ page is unaffected for the default (GENERAL/guest) case — this is what NFR-2 (Section 5.2) requires and is the concrete regression gate for this change.

Chapter 10 — Results and Discussion

The completed feature delivers exactly the behaviour defined in the objectives (Section 3.1). For a representative FAQ corpus tagged across all five phases, each student now sees precisely the categories/content appropriate to their own progress, as summarized in the visibility truth table (Table 7.3) and independently confirmed by the automated test suite (Table 9.2).

10.1 Objective-by-Objective Outcome

Objective (Section 3.1)	Outcome
1. Tag every FAQ with a phase	Delivered — phase field on FAQ, defaulting to GENERAL, editable from the existing admin FAQ form.
2. Track every student's current phase	Delivered — InternshipProgress collection, one record per student, defaulting new/absent students to GENERAL.
3. Automatically filter the FAQ list per student	Delivered — enforced server-side, at the query level, in both listing endpoints.
4. Minimal, additive integration	Delivered — zero changes required to FAQPage.tsx's search/filter/pagination logic.
5. Simple admin interface for phase management	Delivered — Internship Phase tab with filter + edit modal, audit-logged.
6. Compose correctly with batch/program scoping	Delivered and verified — see Test T7, Section 9.4.

Table 10.1 — Objectives versus delivered outcome.

10.2 Discussion

The most important architectural outcome is not the feature's visible behaviour but where the change was made: entirely inside the FAQ query construction, alongside the platform's existing program-scoping mechanism, rather than as a separate filtering layer in the frontend or as a new API surface the frontend had to learn to call differently. This is why the change is transparent to the rest of the platform — search, category filters, pagination, and the FAQ page layout continue to function exactly as they did before this feature was introduced.

The choice to enforce visibility at the database query (rather than filtering an already-fetched result set in application code) also has a performance benefit that compounds as the FAQ corpus grows: MongoDB only ever reads and returns documents the student is entitled to see, so pagination counts (`countDocuments`) and grouped listings remain accurate and efficient without an extra in-memory filter-then-slice step.

The composition test (T7) also surfaced, by design, a reassurance rather than a bug: because `getVisiblePhases()` only ever returns phase values and is combined with the existing `withProgramScope()` output via a plain object merge, there was no code path by which the two scopes could interfere with one another — the two concerns are orthogonal fields (`batchId` and `phase`) ANDed in the same Mongo filter object.

Chapter 11 — Challenges Faced

11.1 Correctly Handling the Two Boundary Phases

The most error-prone part of the phase-visibility rule is its two boundary cases: the very first phase (GENERAL/no record) and the very last (COMPLETED). An earlier, simpler formulation using a single `PHASE_ORDER.slice(currentIndex)` for every input was tempting but wrong — it would have let a Phase 2 student see COMPLETED-only closure content, which is not yet relevant to them. Writing out the full truth table (Table 7.3) before writing the function, and then writing the test suite from that same table, caught this class of off-by-one error before it reached the codebase.

11.2 Choosing Where to Put the Auth Change

The FAQ listing routes were previously fully public with no auth middleware at all. Making them phase-aware required identifying the caller when possible, but the requirement that guests must keep working (FR-4) ruled out simply adding the platform's strict protect middleware, which would have started rejecting anonymous FAQ requests — a regression against a core public-browsing use case. Reusing the platform's existing `protectOptional` middleware (already established elsewhere for an identical "identify-if-possible, never block" need) avoided introducing a second, subtly different auth primitive to reason about.

11.3 Avoiding Duplicate or Divergent Filtering Logic

With two separate FAQ-listing endpoints (grouped and paginated) that both needed the same phase filter, the risk was implementing the lookup-and-filter logic twice and having the two copies quietly drift apart over time (for example, one endpoint being updated for a future phase-related change and the other being missed). Extracting `getVisiblePhases()` as the single source of truth, and adding the `InternshipProgress` lookup as an identical three-line block in both controllers, keeps the two endpoints provably consistent — and is validated directly by running the same test assertions against `getAllFAQs` and would apply identically to `getPaginatedFAQs`.

11.4 Testing Without a Live Database

Because the feature's correctness depends on real MongoDB query semantics (\$in against an enum field, composed with another \$in/equality clause from program scoping), a purely mocked test would have had limited value. Adopting mongodb-memory-server for the test suite added a small amount of setup complexity (a 120-second startup timeout for the in-memory Mongo binary) in exchange for tests that exercise the genuine query path — a trade-off judged worthwhile given how boundary-sensitive this feature's logic is.

Chapter 12 — Conclusion and Future Enhancements

12.1 Conclusion

Phase-Aware FAQ Personalization turns PragatiFAQ's FAQ page from a static, one-size-fits-all list into a personalized view that adapts to each student's actual progress through the Vicharanashala internship. By tagging FAQs with a phase and tracking each student's current phase in a small, purpose-built collection, the platform now automatically surfaces only what is relevant to a given student — reducing clutter, keeping the FAQ page trustworthy as the corpus grows across cohorts, and doing so without disturbing any of PragatiFAQ's existing search, filtering, or administrative workflows.

The feature was delivered as five new files and nine modified files (Table 8.1), required no new infrastructure or third-party dependency, is enforced entirely server-side at the database query level, and is backed by an automated integration test suite covering every phase state and its interaction with the platform's existing program-scoping mechanism. It directly demonstrates the platform's broader design philosophy — extending an existing, proven scoping pattern (as used for batch/program) to a new dimension (student progress) rather than inventing a parallel mechanism.

12.2 Future Enhancements

12.2.1 Automatic Phase Advancement

Phase is currently a manually administered field. A natural next step is driving a student's currentPhase from real milestone events already tracked elsewhere on the platform — for example a report-submission or evaluation sign-off event advancing the student's phase automatically, with the manual admin override in this report retained as an escape hatch for exceptions rather than the primary mechanism.

12.2.2 Using phaseOrder for Fine-Grained Sorting

The FAQ schema already reserves a phaseOrder numeric field (Section 7.1.1), indexed but not yet consumed by any query in this release. A follow-up iteration could use it to control display order of FAQs within a phase bucket (e.g. showing foundational questions before edge-case ones), without requiring any further schema change.

12.2.3 Phase-Aware Search Ranking

The current implementation narrows the candidate set for the FAQ list endpoints but does not yet apply the same phase filter to the platform's hybrid semantic/keyword search endpoint. Weighting or filtering search results toward a student's current phase when a query matches FAQs across multiple phases would extend the same personalization principle to the search experience.

12.2.4 Bulk Phase-Tagging Tools

As the FAQ corpus grows, retroactively classifying a large existing set of untagged or GENERAL-default FAQs one-by-one through the edit form does not scale. A bulk tagging tool (CSV import, or

an AI-assisted suggestion pass using the existing AI-provider infrastructure already used elsewhere on the platform for duplicate detection and auto-answering) would let administrators classify the existing corpus far more quickly.

12.2.5 Student-Facing Phase Indicator

A small, optional enhancement would surface the student's own current phase on the FAQ page itself (e.g. "Showing FAQs for: Phase 2 onward"), giving students visibility into why the list they see differs from what a peer at a different phase sees, and a natural point to contact an administrator if their recorded phase looks wrong.

References

-
- [1] Crowd Source FAQ (PragatiFAQ / Samagama) — Project README, vicharanashala/crowd-source-faq repository.
 - [2] docs/ARCHITECTURE.md — Backend and frontend architecture reference, Crowd Source FAQ repository.
 - [3] docs/PIPELINES.md — Auto-answer, FAQ audit, and freshness pipeline internals, Crowd Source FAQ repository.
 - [4] docs/reference/database-schema.md — Mongoose schema reference, Crowd Source FAQ repository.
 - [5] docs/reference/routes-reference.md and docs/reference/openapi.yaml — API surface reference, Crowd Source FAQ repository.
 - [6] PRODUCT.md — Product overview and technology stack summary, Crowd Source FAQ repository.
 - [7] MongoDB, Inc. — Mongoose ODM documentation (schemas, indexes, enum validators).
 - [8] Express.js documentation — routing and middleware composition.
 - [9] Zod documentation — TypeScript-first schema validation.
 - [10] Vitest documentation — unit and integration testing for TypeScript/ESM projects.
 - [11] mongodb-memory-server documentation — in-memory MongoDB instances for integration tests.

Appendix A — Source File Index

Complete, annotated listings of every new and modified source file are given inline in Chapter 8 (Implementation), at the point each file is introduced, rather than repeated here. This index maps each file to its location in that chapter for quick lookup during review.

File	Documented in
internship.model.ts	Section 8.1.1
phase.helper.ts (getVisiblePhases)	Section 8.1.2
internship.controller.ts	Section 8.1.3
internship.routes.ts	Section 8.1.4
faq.model.ts (phase, phaseOrder fields)	Section 8.2.1
faq.controller.ts (getAllFAQs / getPaginatedFAQs / createFAQ / updateFAQ)	Section 8.2.2

File	Documented in
validation.ts (createFAQSchema / updateFAQSchema)	Section 8.2.3
faq.routes.ts (protectOptional)	Section 8.2.4
admin-log.model.ts (internship_phase_override)	Section 8.2.5
bootstrap/routes.ts	Section 8.2.6
AdminInternshipPhaseTab.tsx	Section 8.3.1
AdminFAQs.tsx	Section 8.3.2
faq-phase-scoping.test.ts	Chapter 9

Table A.1 — Source file index cross-referenced to Chapter 8 and Chapter 9.

Appendix B — Complete API Endpoint Summary

Endpoint	Method	Auth	Feature role
/api/faq	GET	protectOptional	Grouped FAQ list — phase-scoped
/api/faq/paginated	GET	protectOptional	Paginated FAQ list — phase-scoped
/api/faq	POST	protect + authorize(admin, moderator)	Create FAQ — accepts phase
/api/faq/:id	PUT	protect + authorize(admin, moderator)	Update FAQ — accepts phase
/api/admin/internship	GET	protect + adminOnly	List every student + current phase
/api/admin/internship/:userId	PUT	protect + adminOnly	Set a student's current phase (audit-logged)

Table B.1 — Every API endpoint touched by, or introduced for, this feature.

Appendix C — Glossary

PragatiFAQ / Samagama — The internal names for the Crowd Source FAQ platform built for the Vicharanashala Internship Program student community at IIT Ropar.

Batch / Program — An existing scoping dimension representing a cohort, term, or program under which FAQs, categories, and analytics are grouped.

Phase — The new scoping dimension introduced by this feature — one of GENERAL, PHASE_1, PHASE_2, PHASE_3, COMPLETED — representing a stage of the internship.

InternshipProgress — The new Mongoose collection storing one currentPhase value per student.

protectOptional — Existing platform auth middleware that attaches req.user when a valid JWT is present but allows the request to continue anonymously otherwise.

adminOnly — Existing platform middleware shorthand for protect + authorize('admin', 'moderator').

AdminLog — The platform's existing audit-log collection; every administrative action, including phase overrides, is recorded here.

RRF (Reciprocal Rank Fusion) — The platform's existing method for merging vector-search and keyword-search result rankings (unrelated to this feature, noted for platform context in Chapter 4).